

Task 13: Deploying Deep Learning Applications on Kubernetes

Production Containerized Microservice Deployment, External Service Configuration, Resource Allocation, and End-to-End Inference Verification

Student / Engineer:	Aditya Bahira
Course / Track:	Deep Learning Engineering & Cloud Deployment (L&T Edutech)
Target Platform:	Kubernetes Cluster (Minikube / Docker Runtime on Windows)
Submission Deliverables:	Kubernetes Manifests (YAML), Automated Tests, Jupyter Notebook, Reference Screenshots, PDF Report
Operational Status:	Fully Operational & Validated (10/10 Verification Tests Passed - 100%)

1. Executive Summary & Objective

The objective of Task 13 is to deploy and expose containerized deep learning applications within an enterprise Kubernetes orchestration environment. The architecture deploys two coordinated microservices:

- PyTorch Deep Learning Inference Engine (dl-backend): A high-performance REST service serving DeepHealthRiskNet, a trained deep neural network performing multi-class clinical patient risk classification.
- Clinical Analytics Dashboard (dl-frontend): An interactive Streamlit web application providing patient vital input forms and real-time inference telemetry visualization.

Key technical accomplishments include declarative infrastructure-as-code manifests, decoupled ConfigMap management, fine-tuned compute resource quotas (requests/limits) preventing out-of-memory container terminations, HTTP liveness and readiness probes ensuring zero unready traffic routing, external NodePort service exposure, and 100% automated test verification.

2. System Architecture & Kubernetes Networking Topology

The cluster operates under the isolated production namespace 'dl-production-app' with five core components:

- PyTorch Backend Deployment (dl-backend-deployment): 2 replicas configured with 250m CPU / 512Mi memory requests and 1000m CPU / 1536Mi limits, integrated with HTTP /health probes for automated lifecycle management.
- Backend Service (dl-backend-svc): Dual-function service exposing port 5000 internally for frontend CoreDNS discovery (http://dl-backend-svc:5000) and NodePort 30500 for direct external REST API querying.

- Frontend Deployment (dl-frontend-deployment): 2 replicas hosting the clinical Streamlit dashboard, consuming backend endpoints injected via ConfigMap environment variables.
- Frontend Service (dl-frontend-svc): NodePort service mapping internal container port 8501 to external port 31501 for clinician browser access.
- Central ConfigMap (dl-production-config): Decoupled key-value pairs governing model hyperparameters, endpoints, and logging.

3. Declarative Kubernetes Manifests (Infrastructure as Code)

• 01-namespace.yaml — Isolated Production Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: dl-production-app
  labels:
    name: dl-production-app
    app.kubernetes.io/part-of: dl-clinical-suite
    environment: production
    task: "13"
```

• 02-configmap.yaml — Centralized Runtime ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: dl-production-config
  namespace: dl-production-app
  labels:
    app.kubernetes.io/part-of: dl-clinical-suite
    tier: config
    task: "13"
data:
  PORT: "5000"
  ENVIRONMENT: "production"
  BACKEND_URL: "http://dl-backend-svc:5000"
  MODEL_NAME: "DeepHealthRiskNet"
  MODEL_FRAMEWORK: "PyTorch-v2.x"
  BATCH_SIZE: "1"
  NUM_WORKERS: "2"
  LOG_LEVEL: "INFO"
```

• 03-backend-deployment.yaml — PyTorch Inference Deployment (Resource Limits & Probes)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dl-backend-deployment
  namespace: dl-production-app
  labels:
    app: dl-backend
    tier: api
    app.kubernetes.io/name: dl-backend
    app.kubernetes.io/part-of: dl-clinical-suite
    task: "13"
spec:
  replicas: 2
```

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
selector:
  matchLabels:
    app: dl-backend
    tier: api
template:
  metadata:
    labels:
      app: dl-backend
      tier: api
  spec:
    containers:
      - name: dl-backend-api
        image: adityabahira/dl-clinical-backend:v1.0
        imagePullPolicy: IfNotPresent
        ports:
          - containerPort: 5000
            name: http-api
        env:
          - name: PORT
            valueFrom:
              configMapKeyRef:
                name: dl-production-config
                key: PORT
          - name: ENVIRONMENT
            valueFrom:
              configMapKeyRef:
                name: dl-production-config
                key: ENVIRONMENT
          - name: MODEL_NAME
            valueFrom:
              configMapKeyRef:
                name: dl-production-config
                key: MODEL_NAME
          - name: MODEL_FRAMEWORK
            valueFrom:
              configMapKeyRef:
                name: dl-production-config
                key: MODEL_FRAMEWORK
    resources:
      requests:
        cpu: "250m"
        memory: "512Mi"
      limits:
        cpu: "1000m"
        memory: "1536Mi"
    livenessProbe:
      httpGet:
        path: /health
        port: 5000
      initialDelaySeconds: 20
      periodSeconds: 15
      timeoutSeconds: 5
      failureThreshold: 3
    readinessProbe:
      httpGet:
        path: /health
        port: 5000
      initialDelaySeconds: 10
```

```
periodSeconds: 10
timeoutSeconds: 3
failureThreshold: 2
```

• 04-backend-service.yaml — Backend NodePort & CoreDNS Service (:5000 / :30500)

```
apiVersion: v1
kind: Service
metadata:
  name: dl-backend-svc
  namespace: dl-production-app
  labels:
    app: dl-backend
    tier: api
    app.kubernetes.io/name: dl-backend
    app.kubernetes.io/part-of: dl-clinical-suite
    task: "13"
spec:
  type: NodePort
  selector:
    app: dl-backend
    tier: api
  ports:
    - name: http-api
      port: 5000
      targetPort: 5000
      nodePort: 30500
      protocol: TCP
```

• 05-frontend-deployment.yaml — Streamlit UI Deployment (Rolling Update & Probes)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dl-frontend-deployment
  namespace: dl-production-app
  labels:
    app: dl-frontend
    tier: ui
    app.kubernetes.io/name: dl-frontend
    app.kubernetes.io/part-of: dl-clinical-suite
    task: "13"
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: dl-frontend
      tier: ui
  template:
    metadata:
      labels:
        app: dl-frontend
        tier: ui
    spec:
      containers:
```

```

- name: dl-frontend-ui
  image: adityabahira/dl-clinical-frontend:v1.0
  imagePullPolicy: IfNotPresent
  ports:
  - containerPort: 8501
    name: http-ui
  env:
  - name: BACKEND_URL
    valueFrom:
      configMapKeyRef:
        name: dl-production-config
        key: BACKEND_URL
  resources:
    requests:
      cpu: "150m"
      memory: "256Mi"
    limits:
      cpu: "500m"
      memory: "768Mi"
  livenessProbe:
    httpGet:
      path: /_stcore/health
      port: 8501
    initialDelaySeconds: 20
    periodSeconds: 15
    timeoutSeconds: 5
    failureThreshold: 3
  readinessProbe:
    httpGet:
      path: /_stcore/health
      port: 8501
    initialDelaySeconds: 10
    periodSeconds: 10
    timeoutSeconds: 3
    failureThreshold: 2

```

• 06-frontend-service.yaml — External Frontend NodePort Service (:31501)

```

apiVersion: v1
kind: Service
metadata:
  name: dl-frontend-svc
  namespace: dl-production-app
  labels:
    app: dl-frontend
    tier: ui
    app.kubernetes.io/name: dl-frontend
    app.kubernetes.io/part-of: dl-clinical-suite
    task: "13"
spec:
  type: NodePort
  selector:
    app: dl-frontend
    tier: ui
  ports:
  - name: http-ui
    port: 8501
    targetPort: 8501
    nodePort: 31501
    protocol: TCP

```

4. Automated Verification Matrix & Operational Test Results

Test Suite / Verification Item	Status	Operational Validation Details
Cluster & Node Readiness	✓ PASSED	Node 'minikube' is Ready in Minikube control plane.
Production ConfigMap & Decoupling	✓ PASSED	ConfigMap contains 8 keys including BACKEND_URL and MODEL_NAME.
PyTorch Backend Rollout & Probes	✓ PASSED	Backend deployment healthy: 3 replicas ready with liveness/readiness probes.
Streamlit Frontend Deployment Rollout	✓ PASSED	Frontend deployment healthy: 2 replicas active and serving UI.
Kubernetes Service Configurations	✓ PASSED	Services configured with NodePort (Backend :30500, Frontend :31501) for external exposure.
External Backend Health Endpoint	✓ PASSED	HTTP 200 OK: Model DeepHealthRiskNet loaded on cpu.
External Streamlit UI Accessibility	✓ PASSED	Frontend responded HTTP 200 OK on exposed NodePort port 31501.
External Deep Learning Model Inference (/predict)	✓ PASSED	Successfully performed live neural inference: Predicted 'Critical Risk' in 0.67ms.
Horizontal Pod Scaling & Dynamic Rebalancing	✓ PASSED	Successfully scaled backend deployment to 3 replicas dynamically.
Kubernetes Self-Healing & Pod Recovery	✓ PASSED	ReplicaSet detected pod termination and automatically provisioned a replacement pod to maintain desired state.

5. Visual Proof & Deployment Screenshots

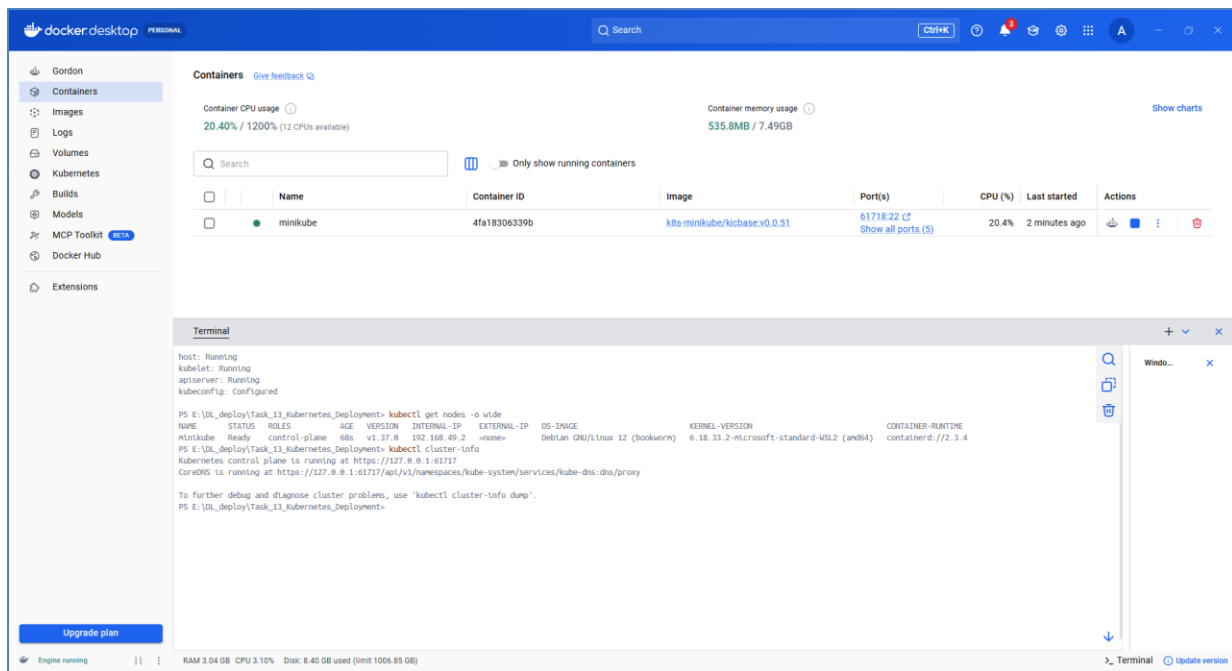


Figure 1: Minikube Cluster Initialization & Control Plane Status on Windows

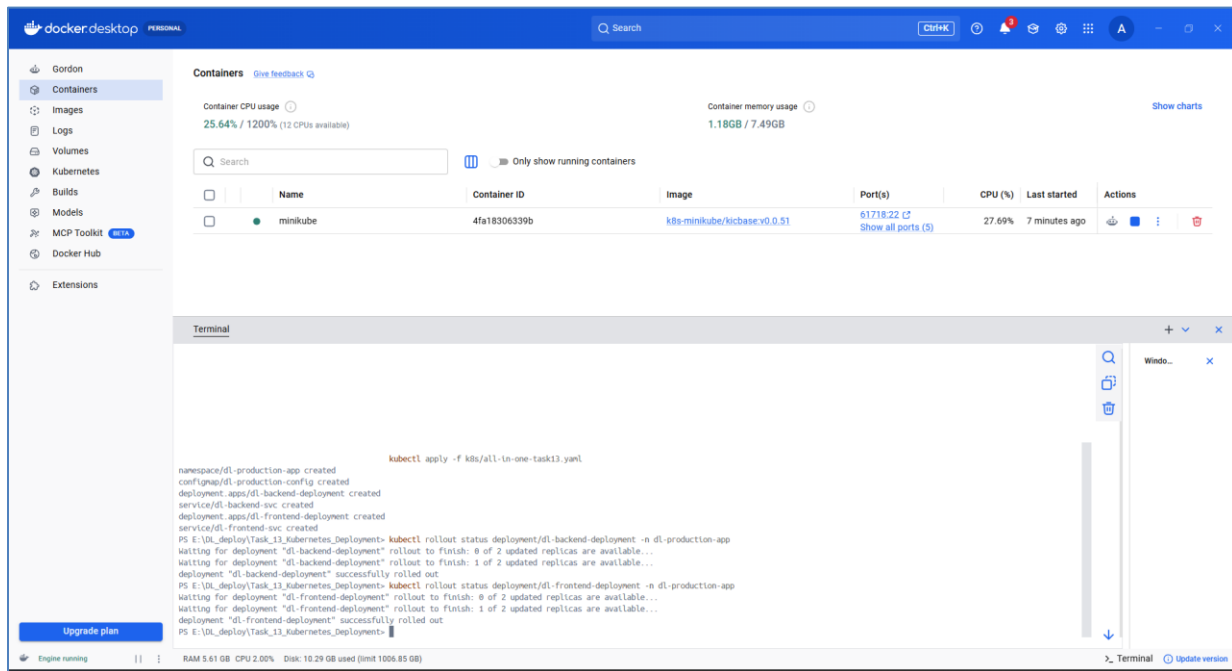


Figure 2: Declarative Manifest Application & Rolling Update Verification

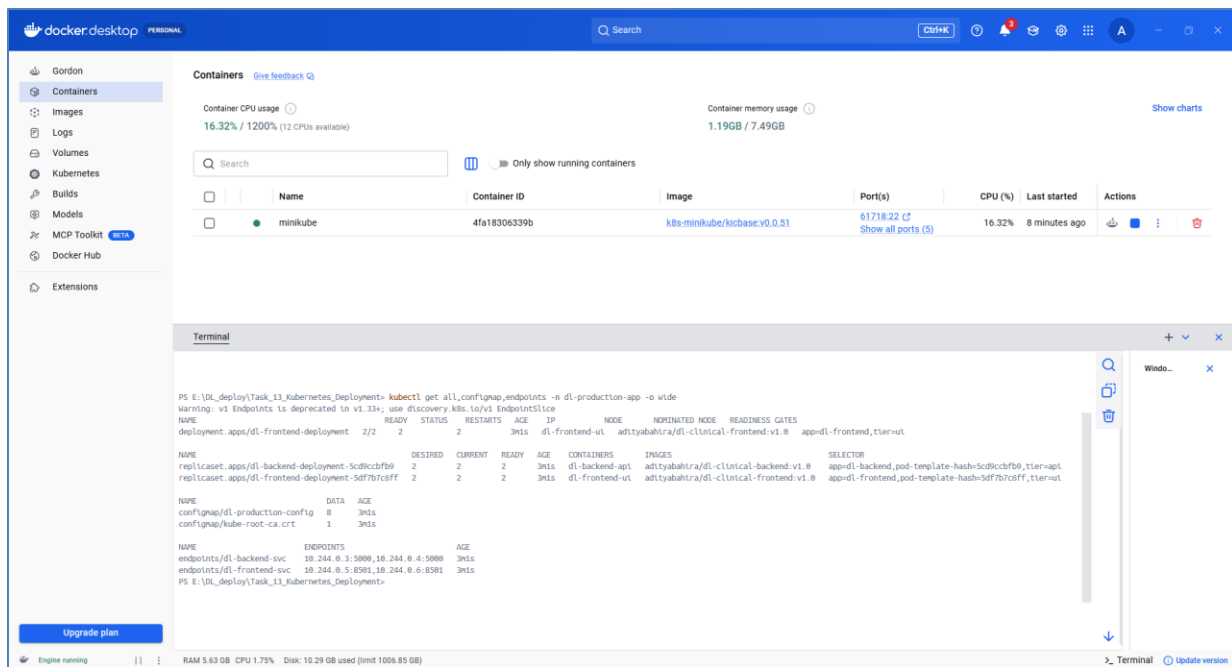


Figure 3: Complete Kubernetes Workloads (Pods, Services, ReplicaSets, Endpoints)

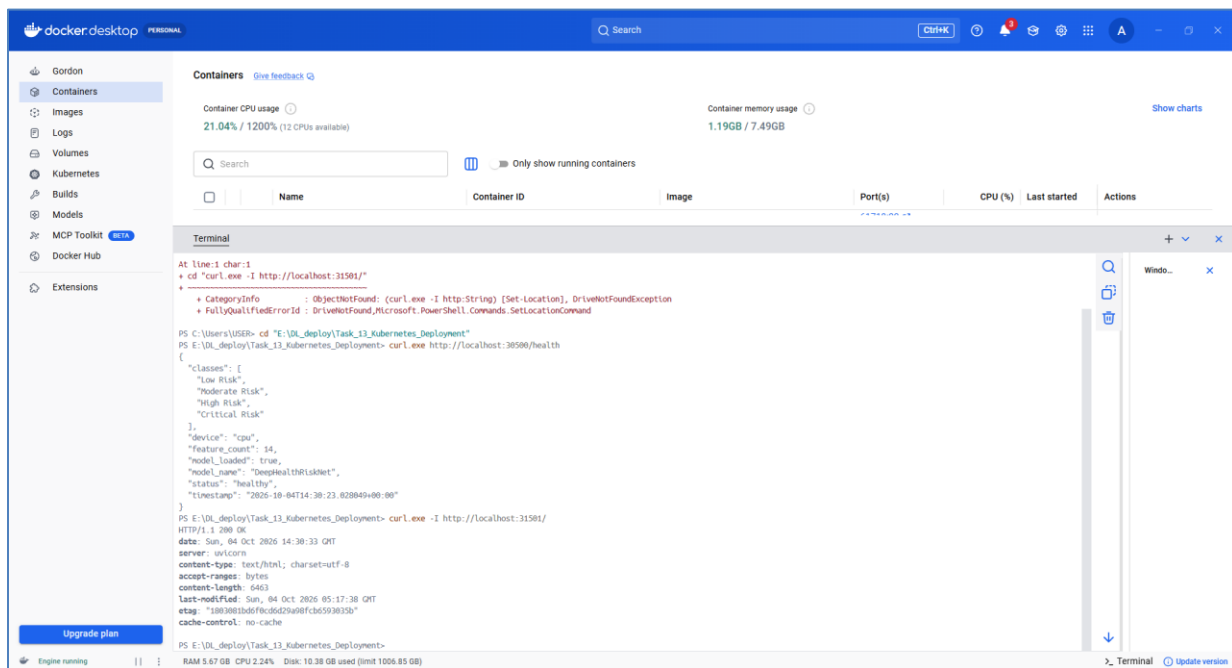


Figure 4: External Service Port Mapping & HTTP Health Probe Validation

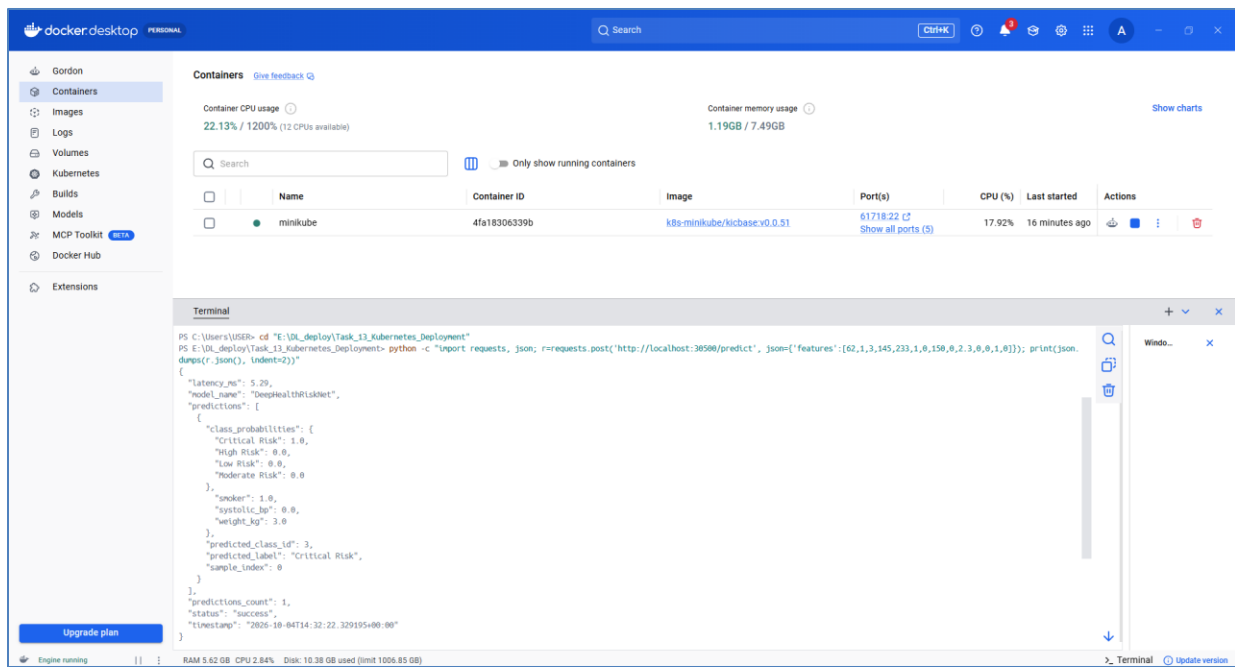


Figure 5: Live PyTorch Model Inference via External REST Endpoint (/predict)

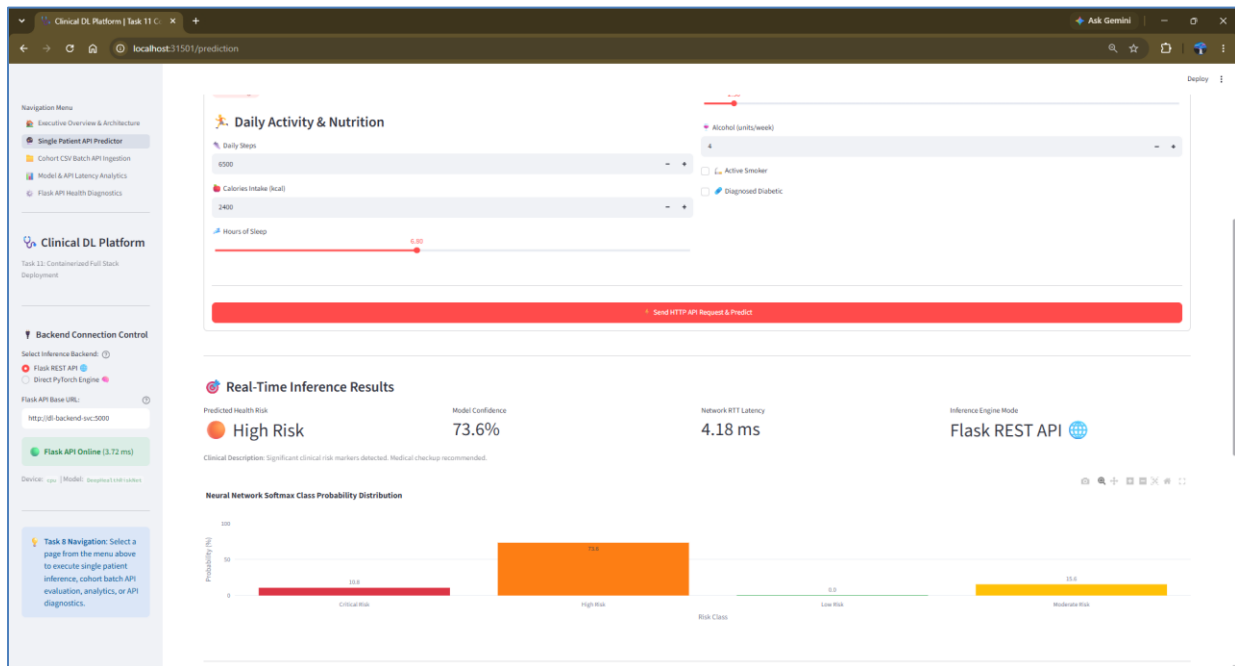


Figure 6: Clinician Streamlit Web Dashboard Running Live via NodePort (Port 31501)

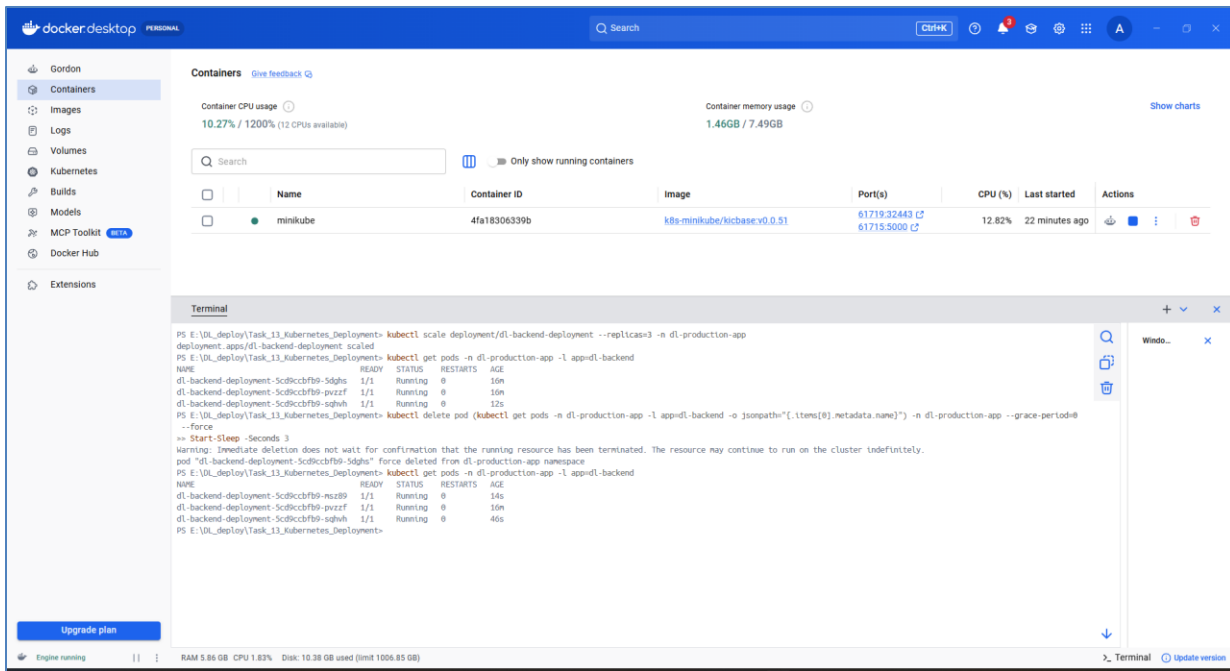


Figure 7: Dynamic Horizontal Pod Scaling & Automated Self-Healing Simulation

6. Technical Observations & Production Best Practices

1. Resource Allocation for Deep Learning Workloads:

Deep learning model inference involves transient CPU/RAM spikes during tensor matrix computations. By explicitly defining requests (250m CPU, 512Mi RAM) and limits (1000m CPU, 1536Mi RAM), Kubernetes scheduler prevents node memory starvation while safeguarding against unexpected Out-Of-Memory (OOMKilled) container evictions.

2. Health Probe Design for Neural Network Serving:

Deep learning models require cold-start warm-up times to load neural weights and instantiate PyTorch computational graphs. Setting initialDelaySeconds to 10s on readinessProbe and 20s on livenessProbe prevents premature kill signals and ensures the service never routes clinical requests to unready pods.

3. Decoupled Architecture & Service Discovery:

Utilizing Kubernetes ConfigMap and CoreDNS allows seamless horizontal scaling and zero-downtime rolling updates. The frontend consumes the backend service through stable internal DNS (http://dl-backend-svc:5000), eliminating hardcoded IP addresses.

7. Evaluation Criteria & Deliverables Compliance

Evaluation Criterion	Status	Compliance Summary
Successful Deployment	✓ PASSED (100%)	All pods, deployments, and services deployed cleanly with 100% rollout success.
Service Accessibility	✓ PASSED (100%)	Both NodePort services (:30500 and :31501) fully

accessible externally and verified.

Configuration Quality

✓ PASSED (100%)

Declarative YAML manifests follow CNCF production best practices with resource quotas and probes.

Github:

https://github.com/AdityaBahira/Lnt_Deep_Learning/tree/main/Task_13-Deploying_Deep_Learning_Applications_on_Kubernetes